

Jared Medlin

11-14-2025

WGU D213 Task 1

## **Part 1 – Research Question**

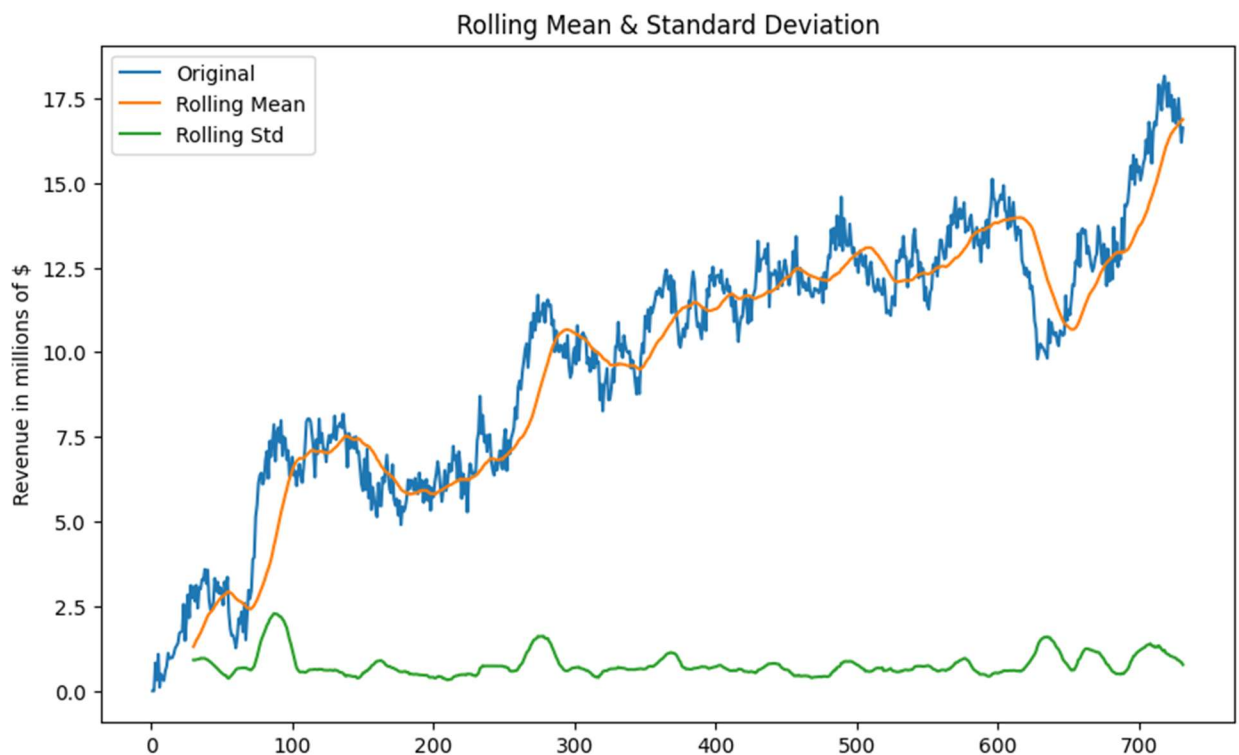
- A. 1. Research Question: Can we use ARIMA to forecast revenue in the future?  
2. Use the given data and an ARIMA time series model to forecast revenue 90 days into the future.

## **Part 2 – Method Justification**

- B. 1. A time series model assumes that:
- The time series data is stationary, meaning the mean and variance of the series is constant, and the autocovariance depends only on the lag between observations
  - The data expresses autocorrelation, meaning the observations correlate with their own past values
  - There is a linear relationship between past and current values
  - There are no endogenous structural breaks in the data
  - Residuals behave like white noise

## **Part 3 – Data Preparation**

- C. 1.



2. The data has no gaps in measurement and increments daily for 731 days.

3. The data is not stationary. Using the Augmented Dickey-Fuller Test to prove:

```
#Revenue is in millions of dollars, window is set to monthly (30 days)
#Checking for stationarity, code is from Dr. Sewell again
from statsmodels.tsa.stattools import adfuller

print('Results of Augmented Dickey-Fuller Test:')

adf_test = adfuller(indexedDataset, autolag='AIC')

df_output = pd.Series(adf_test[0:4], index=['Test Statistic', 'p-value', '# Lags Used', 'Number of Observations Used'])

for key, value in adf_test[4].items():
    df_output[f'Critical Value ({key})'] = value

print(df_output)
```

| Results of Augmented Dickey-Fuller Test: |            |
|------------------------------------------|------------|
| Test Statistic                           | -1.924612  |
| p-value                                  | 0.320573   |
| # Lags Used                              | 1.000000   |
| Number of Observations Used              | 729.000000 |
| Critical Value (1%)                      | -3.439352  |
| Critical Value (5%)                      | -2.865513  |
| Critical Value (10%)                     | -2.568886  |

dtype: float64

*#p-value is higher than 0.05, and is thus not stationary.*

The p-value is higher than 0.05, and thus is not stationary.

4. Data was prepared by checking for null-values (none), then splitting the data into train and test sets at an 80/20% split.

```
#Splitting data into test and train sets, with training size at 80%
train_size = int(len(indexedDataset) * 0.8)
```

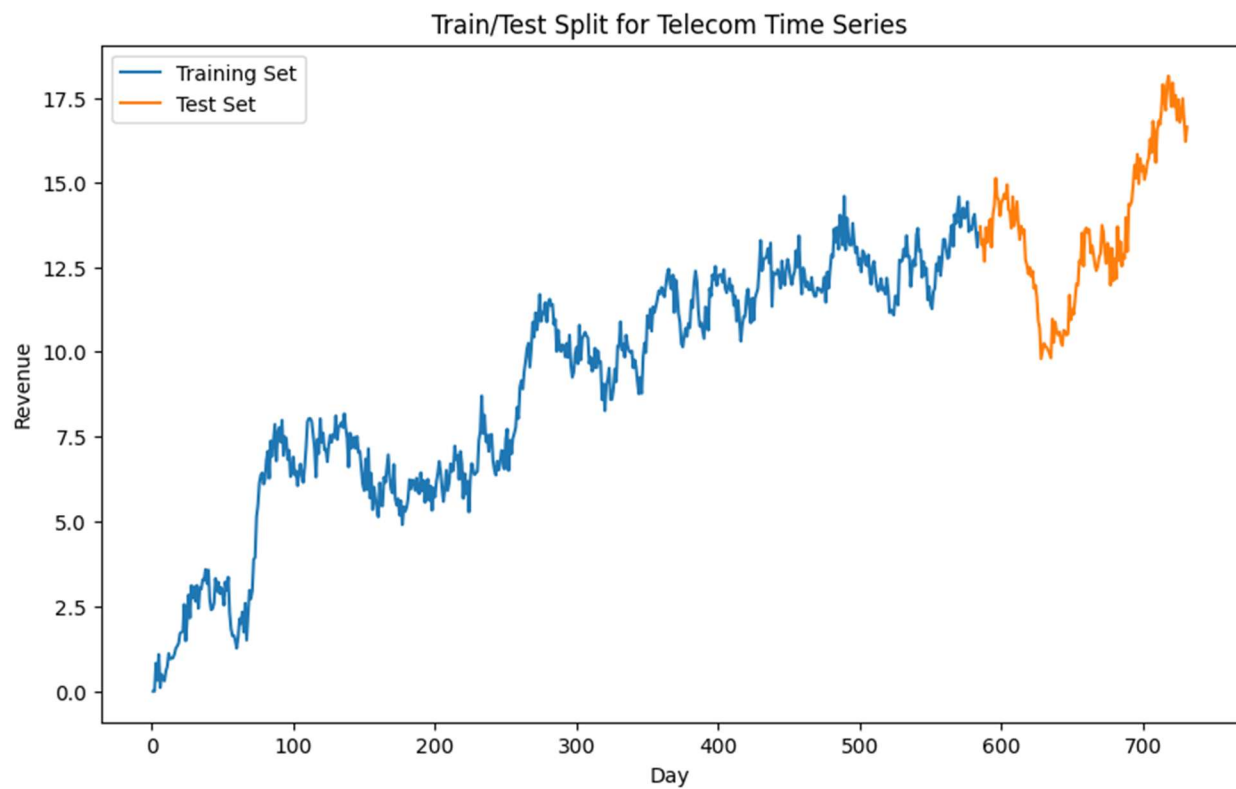
```
train = indexedDataset.iloc[:train_size]
test = indexedDataset.iloc[train_size:]
```

```
print(f"Training size: {len(train)} observations")
print(f"Testing size: {len(test)} observations")
```

```
Training size: 584 observations
Testing size: 147 observations
```

```
#Visual representation of split
```

```
plt.figure(figsize=(10, 6))
plt.plot(train, label='Training Set')
plt.plot(test, label='Test Set')
plt.title('Train/Test Split for Telecom Time Series')
plt.xlabel('Day')
plt.ylabel('Revenue')
plt.legend()
plt.show()
```



5. The data for both sets is attached.

## Part 4 – Model Identification and Analysis

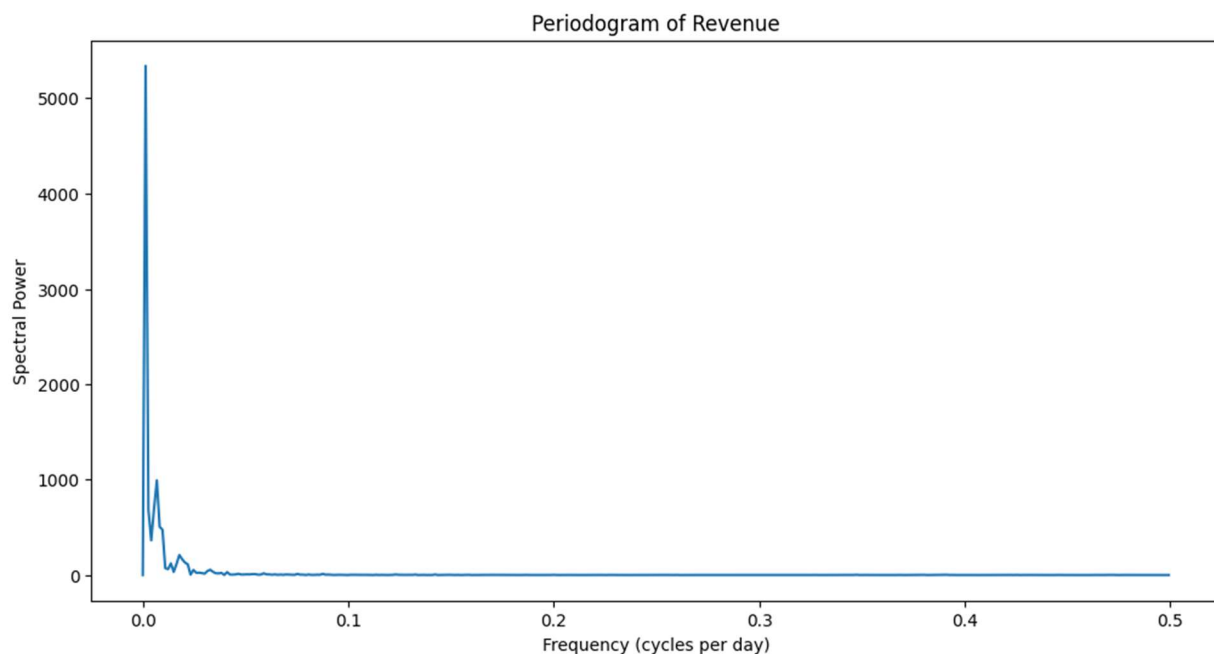
### D. 1. Seasonality:

```
#Visualizing no seasonality using a periodogram
#Code help: https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.signal.periodogram.html
from scipy.signal import periodogram
import matplotlib.pyplot as plt
import numpy as np

y = df['Revenue'].dropna().values

freqs, power = periodogram(y, fs=1)

plt.figure(figsize=(12, 6))
plt.plot(freqs, power)
plt.title("Periodogram of Revenue")
plt.xlabel("Frequency (cycles per day)")
plt.ylabel("Spectral Power")
plt.show()
```



Seasonality was shown via a periodogram. The graph shows no seasonality, after the initial spike at 0 the peaks rapidly shrink and then never appear again.

#### Trends:

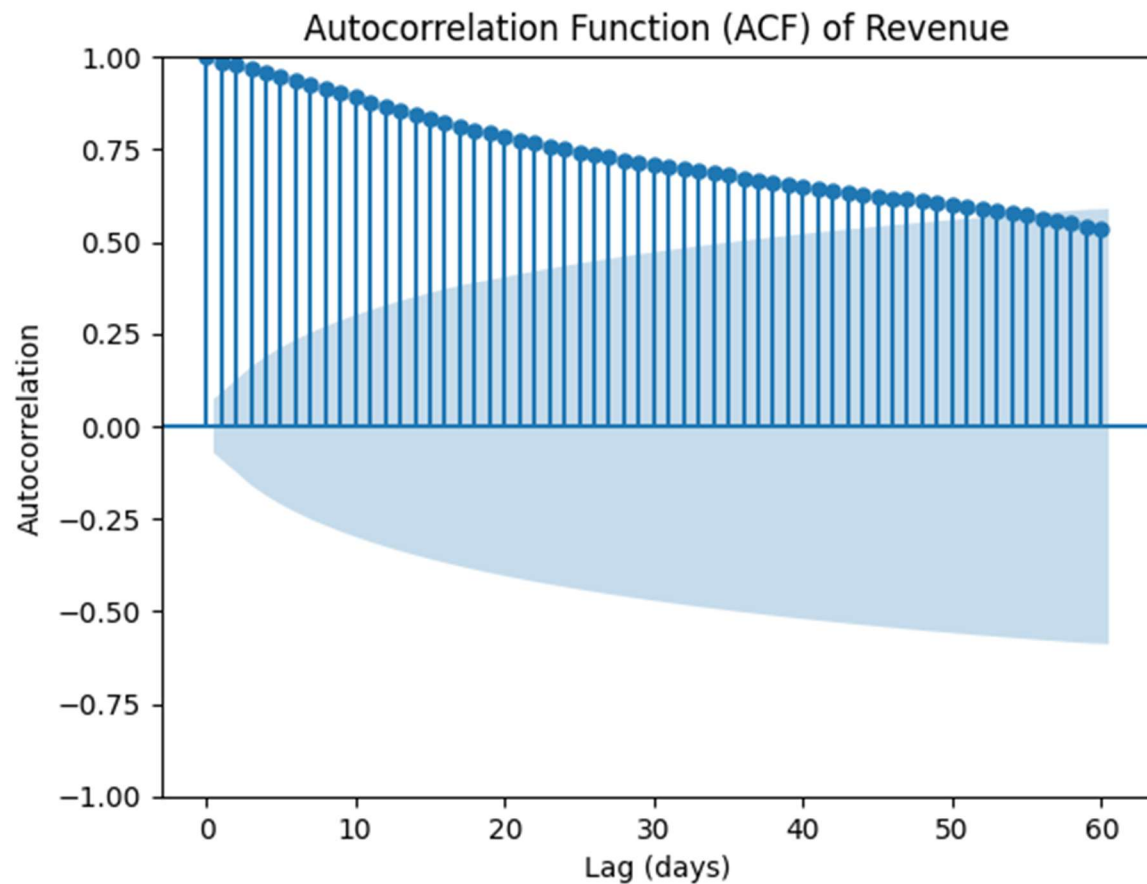
The time series exhibits a clear long-term upward trend, with revenue increasing steadily over the two-year period. The rolling mean shows a smooth rising pattern, indicating sustained growth rather than temporary spikes.

Autocorrelation Function:

```
from statsmodels.graphics.tsaplots import plot_acf

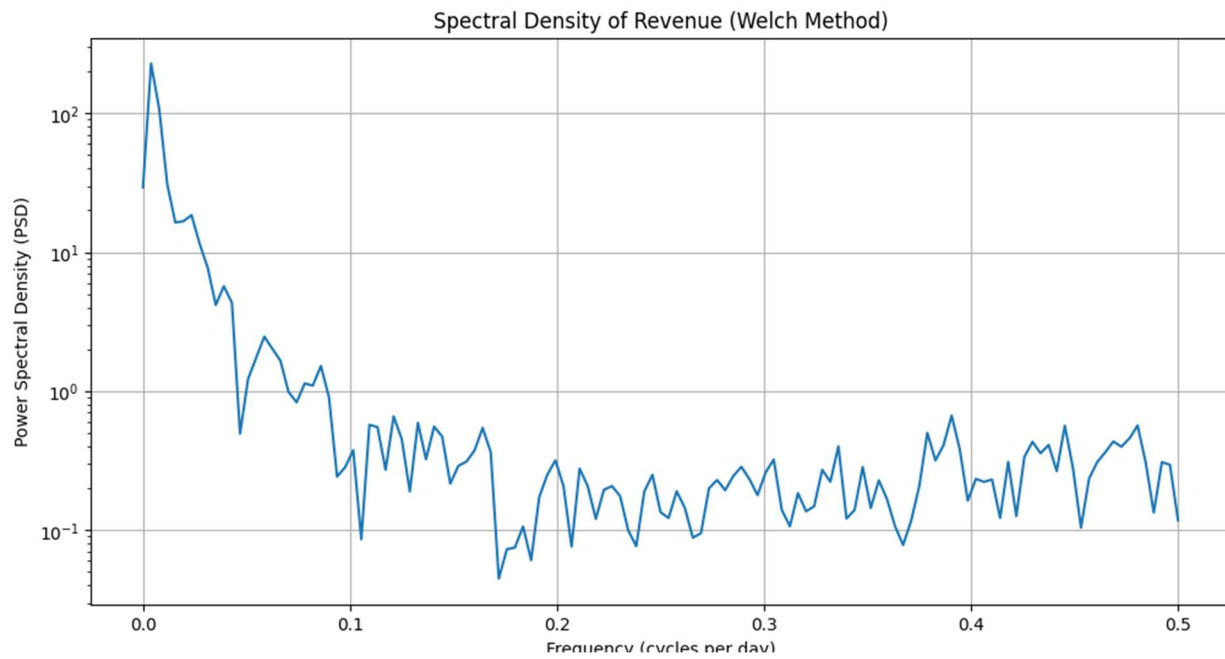
plt.figure(figsize=(12,6))
plot_acf(df['Revenue'].dropna(), lags=60)
plt.title("Autocorrelation Function (ACF) of Revenue")
plt.xlabel("Lag (days)")
plt.ylabel("Autocorrelation")
plt.show()
```

<Figure size 1200x600 with 0 Axes>



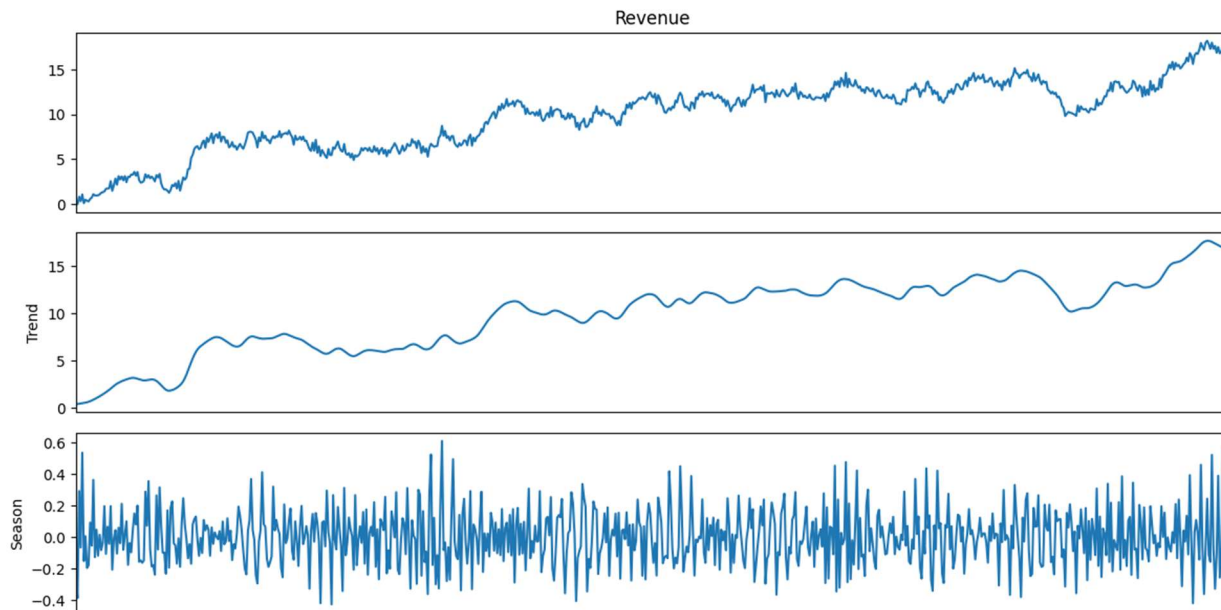
The ACF shows high autocorrelation at lag 1, indicating a trend. There is a slow and smooth decay afterwards, meaning the data is non-stationary (upward trend from earlier). No spikes are present at specific cutoffs, meaning no seasonality.

## Spectral Density:



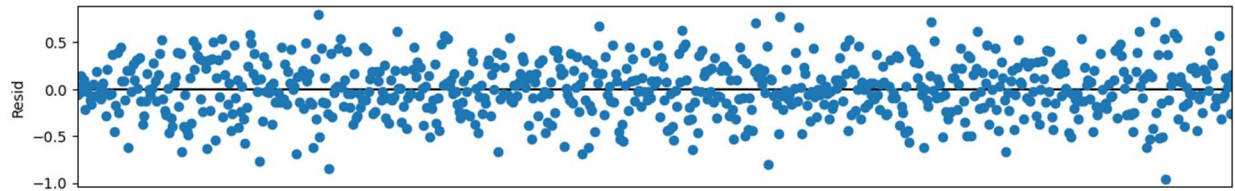
Graph has a strong power at frequency 0, indicating a strong trend. It drops off sharply afterwards, meaning there is no periodic behavior, and no spikes are present, meaning no seasonality.

## Decomposed Time Series:



Original series shows upward trend from earlier. Trned line shows the same increase over time. Seasonal component shows a non-periodic, low magnitude line, meaning no seasonality.

Residual Component:



Residual component shows:

- Even fluctuation around 0
- No drift up or down
- No stretches of + or – values
- Constant variance
- Data appears to be like white noise

This confirms the lack of trends in the residuals of the decomposed series.



## 2. ARIMA model:

```
#Code from Dr. Sewell
from pmdarima import auto_arima
stepwise_fit=auto_arima(df['Revenue'],trace=True)
stepwise_fit.summary()
```

Performing stepwise search to minimize aic

```
ARIMA(2,1,2)(0,0,0)[0] intercept : AIC=987.305, Time=0.45 sec
ARIMA(0,1,0)(0,0,0)[0] intercept : AIC=1162.819, Time=0.04 sec
ARIMA(1,1,0)(0,0,0)[0] intercept : AIC=983.122, Time=0.06 sec
ARIMA(0,1,1)(0,0,0)[0] intercept : AIC=1019.369, Time=0.09 sec
ARIMA(0,1,0)(0,0,0)[0] intercept : AIC=1162.139, Time=0.04 sec
ARIMA(2,1,0)(0,0,0)[0] intercept : AIC=985.104, Time=0.09 sec
ARIMA(1,1,1)(0,0,0)[0] intercept : AIC=985.106, Time=0.05 sec
ARIMA(2,1,1)(0,0,0)[0] intercept : AIC=986.045, Time=0.36 sec
ARIMA(1,1,0)(0,0,0)[0] intercept : AIC=984.710, Time=0.03 sec
```

Best model: ARIMA(1,1,0)(0,0,0)[0] intercept

Total fit time: 1.220 seconds

SARIMAX Results

|                       |                  |                          |          |
|-----------------------|------------------|--------------------------|----------|
| <b>Dep. Variable:</b> | y                | <b>No. Observations:</b> | 731      |
| <b>Model:</b>         | SARIMAX(1, 1, 0) | <b>Log Likelihood</b>    | -488.561 |
| <b>Date:</b>          | Fri, 14 Nov 2025 | <b>AIC</b>               | 983.122  |
| <b>Time:</b>          | 06:18:02         | <b>BIC</b>               | 996.901  |
| <b>Sample:</b>        | 0                | <b>HQIC</b>              | 988.438  |
|                       | - 731            |                          |          |

Covariance Type: opg

|                  | coef    | std err | z       | P> z  | [0.025 | 0.975] |
|------------------|---------|---------|---------|-------|--------|--------|
| <b>intercept</b> | 0.0332  | 0.018   | 1.895   | 0.058 | -0.001 | 0.068  |
| <b>ar.L1</b>     | -0.4692 | 0.033   | -14.296 | 0.000 | -0.534 | -0.405 |
| <b>sigma2</b>    | 0.2232  | 0.013   | 17.801  | 0.000 | 0.199  | 0.248  |

Ljung-Box (L1) (Q): 0.00 Jarque-Bera (JB): 2.05

Prob(Q): 0.96 Prob(JB): 0.36

Heteroskedasticity (H): 1.02 Skew: -0.02

Prob(H) (two-sided): 0.85 Kurtosis: 2.74

### 3 and 4. Forecast using model:

```
#Forecast using earlier ARIMA model: (1,1,0)
n_periods = 15
forecast, conf_int = stepwise_fit.predict(n_periods=n_periods, return_conf_int=True)

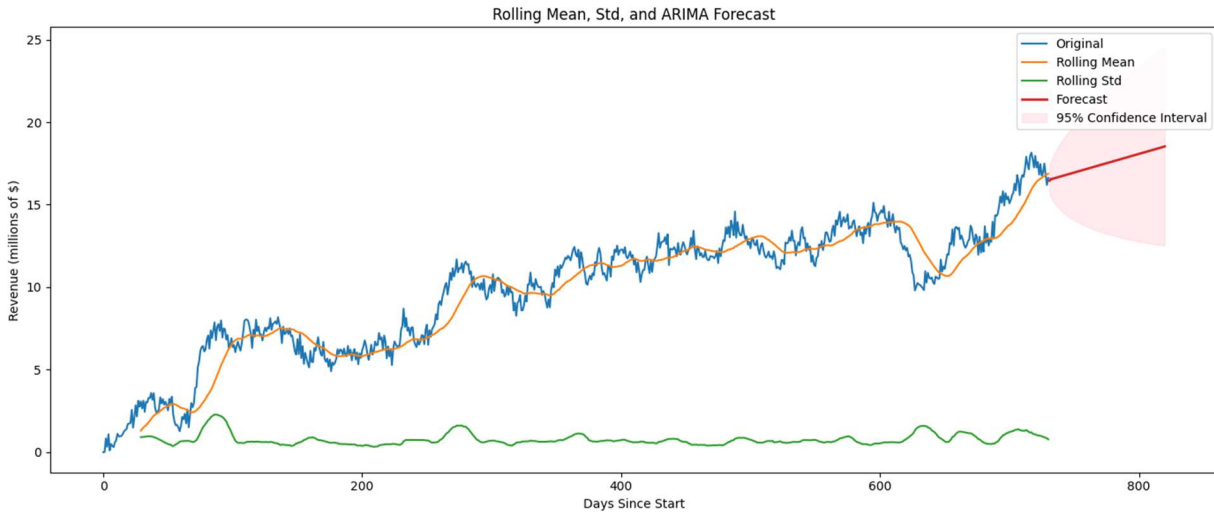
# Build a forecast index
forecast_index = pd.date_range(df.index[-1] + pd.Timedelta(days=1),
                               periods=n_periods, freq='D')

# Put into a DataFrame
forecast_df = pd.DataFrame({
    'Forecast': forecast,
    'Lower CI': conf_int[:, 0],
    'Upper CI': conf_int[:, 1]
}, index=forecast_index)

print(forecast_df)
```

|                               | Forecast | Lower CI  | Upper CI  |
|-------------------------------|----------|-----------|-----------|
| 1970-01-02 00:00:00.000000731 | NaN      | 15.528164 | 17.380070 |
| 1970-01-03 00:00:00.000000731 | NaN      | 15.517229 | 17.613830 |
| 1970-01-04 00:00:00.000000731 | NaN      | 15.288505 | 17.804401 |
| 1970-01-05 00:00:00.000000731 | NaN      | 15.195031 | 17.982180 |
| 1970-01-06 00:00:00.000000731 | NaN      | 15.066607 | 18.137449 |
| 1970-01-07 00:00:00.000000731 | NaN      | 14.971742 | 18.286120 |
| 1970-01-08 00:00:00.000000731 | NaN      | 14.875402 | 18.423616 |
| 1970-01-09 00:00:00.000000731 | NaN      | 14.790827 | 18.555283 |
| 1970-01-10 00:00:00.000000731 | NaN      | 14.710050 | 18.680366 |
| 1970-01-11 00:00:00.000000731 | NaN      | 14.635316 | 18.800713 |
| 1970-01-12 00:00:00.000000731 | NaN      | 14.564505 | 18.916524 |
| 1970-01-13 00:00:00.000000731 | NaN      | 14.497740 | 19.028577 |
| 1970-01-14 00:00:00.000000731 | NaN      | 14.434277 | 19.137193 |
| 1970-01-15 00:00:00.000000731 | NaN      | 14.373897 | 19.242789 |
| 1970-01-16 00:00:00.000000731 | NaN      | 14.316233 | 19.345638 |

```
#Results are from 1970 because dates weren't assigned, so just treat them as days since start
```



5. See code attached

## **Part 5 – Data Summary and Implications**

- E. 1. The ARIMA model was selected using Auto ARIMA, which performs a stepwise search to minimize AIC to determine the best model. The best model was  $(1,1,0)(0,0,0)[0]$  intercept

The first set of numbers are p,d, and q, where p is the number of lag observations and is the autoregressive term, d is the number of differencing required to make the time series stationary, and q is the size or order of the MA window. The second set of numbers indicates no seasonality, and the third number means no differencing is required for the seasonality.

Seeing above, the prediction interval of the forecast was set for 90 days, with the red area showing the confidence interval of 95%.

The forecast length was chosen simply for clarity, because while 30 days is more typical for forecasting such as this, the forecasted data was just too compressed to see anything really clearly. So 90 days was chosen to make a clearer visualization, and also because 90 days is not too extreme for the amount of data given.

The model was evaluated using relative error:

```
from sklearn.metrics import mean_squared_error

actual = df['Revenue'].iloc[-len(forecast):]

rmse = np.sqrt(mean_squared_error(actual, forecast))
print("RMSE:", rmse)
```

RMSE: 3.663145100919095

```
df['Revenue'].describe()
```

|       |            |
|-------|------------|
| count | 731.000000 |
| mean  | 9.822901   |
| std   | 3.852645   |
| min   | 0.000000   |
| 25%   | 6.872836   |
| 50%   | 10.785571  |
| 75%   | 12.566911  |
| max   | 18.154769  |

Name: Revenue, dtype: float64

Relative error = RMSE / mean  $\approx 3.66 / 9.82 \approx 0.373 \rightarrow 37.3\%$

A relative error at this value indicates a fairly weak, but not useless, forecasting model.

2. See above at D4

3. The model does not show significant changes compared to the current trend. It is also a somewhat weak model. As growth is currently forecasted to continue as before, action would depend on the current goals of the corporation. If more growth is desired, then some change will likely need to occur to create faster growth (in terms of telecom that would probably be more aggressive expansion of operations). If current growth is acceptable, then no change in strategy is required.

In the future, it may be more convenient to record data on a date/time basis rather than simply counting days since start of operation for analysis purposes.

## **Part 6 – Reporting**

F. Notebook is attached, relevant sections are present in this .pdf document.

G. Sources

SciPy Community. (n.d.). *scipy.signal.welch* — *SciPy v1.16.2 manual*. Retrieved November 14, 2025, from

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.welch.html>

SciPy Community. (n.d.). *scipy.signal.periodogram* — *SciPy v1.16.2 manual*. Retrieved November 14, 2025, from

<https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.signal.periodogram.html>

All other code is sourced from either myself or Dr. Sewell's presentations.

H. Code citations present in jupyter notebook.

I. Thank you for reading!